



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2
ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Linear Data Structures Stack, Queue, and Linked List

ผู้ช่วยศาสตราจารย์ ดร.ธรรศกัญญา สุระศักดิ์

thattapon.s@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

© Asst. Prof. Thattapon Surasak, Ph.D. All rights reserved.

Outline

1. Introduction to Data Structures
2. Stack
3. Queue
4. Comparison between Stack and Queue
5. Linked List
6. Linked List Operations
7. Stack and Queue using Linked List
8. Summary and Practice



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2 ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

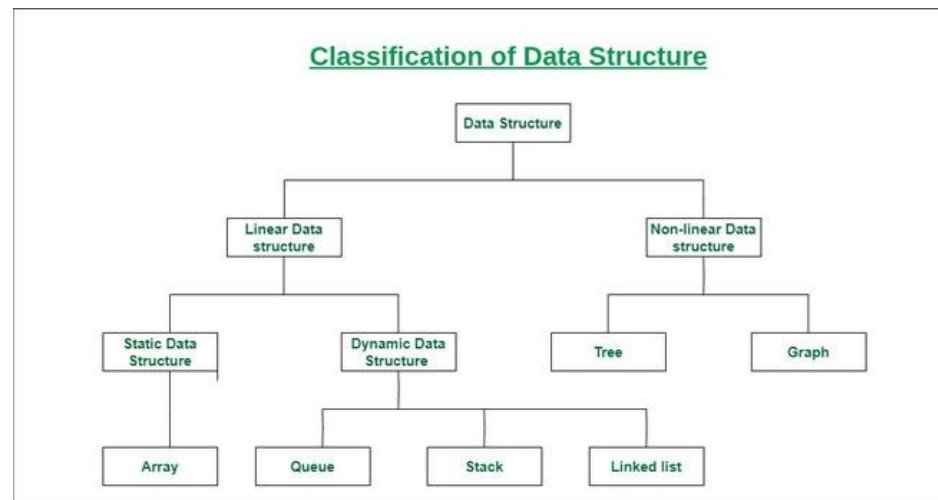
ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Introduction to Data Structures

What is Data Structure?

Data Structure คือ วิธีการจัดเก็บข้อมูลอย่างเป็นระบบเพื่อให้สามารถเข้าถึง ใช้งาน เพิ่ม และลบข้อมูลได้อย่างเหมาะสม





Why Do We Need Data Structures?

เพราะข้อมูลในคอมพิวเตอร์มีจำนวนมากและแต่ละสถานการณ์ต้องการวิธีจัดเก็บที่ต่างกัน

- ถ้าเลือกโครงสร้างข้อมูลเหมาะสม
- โปรแกรมจะทำงานง่ายขึ้น
- โค้ดจะเป็นระบบมากขึ้น
- การประมวลผลจะมีประสิทธิภาพมากขึ้น



Linear Data Structure

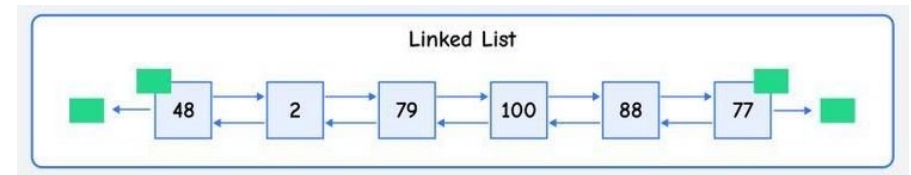
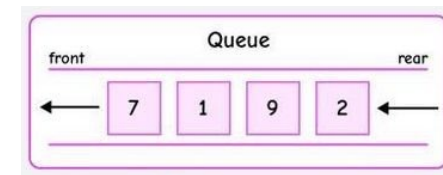
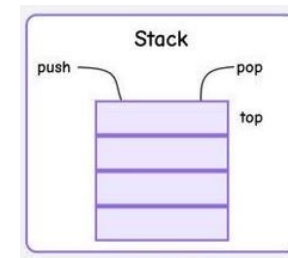
โครงสร้างข้อมูลเชิงเส้น คือ การเก็บข้อมูลเรียงต่อกันเป็นลำดับสมาชิกแต่ละตัวมีตำแหน่งก่อนหน้าและถัดไปอย่างชัดเจน

ตัวอย่าง

- Array
- Stack
- Queue
- Linked List

Everyday Analogy

- **Stack** คือโครงสร้างที่เอาของเข้าและออกด้านเดียว
 - กองหนังสือหรือกองจาน → Stack
- **Queue** คือโครงสร้างที่เข้าท้ายออกหน้า
 - การต่อคิวซื้ออาหาร → Queue
- **Linked List** คือการเก็บข้อมูลแบบเชื่อมต่อกันเป็นข้อ ๆ
 - โซ่ที่เชื่อมต่อกันเป็นข้อ ๆ → Linked List





ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2 ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Section 1: Stack

What is Stack?

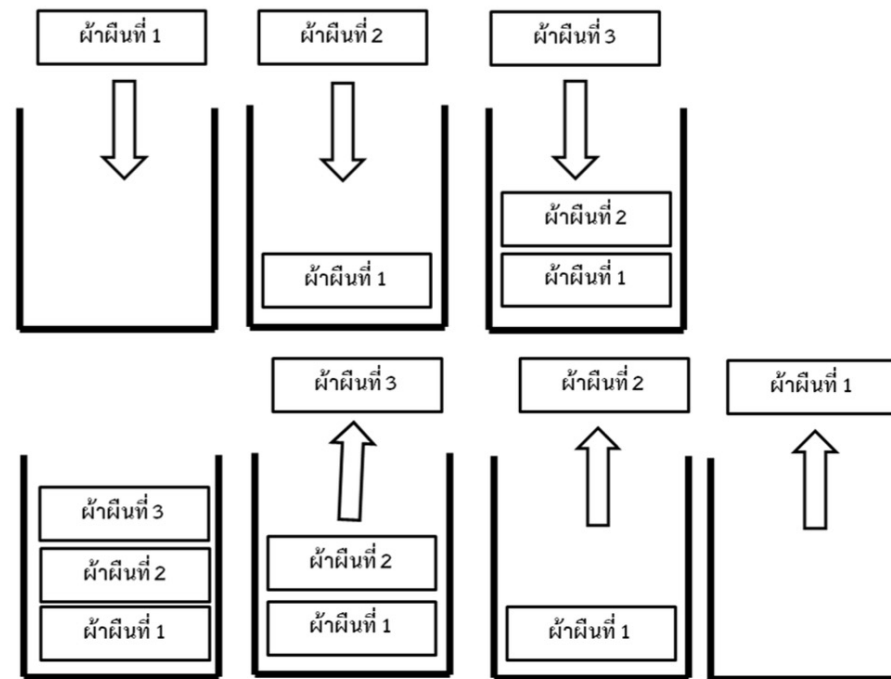
Stack คือ โครงสร้างข้อมูลทำงานแบบ

“เข้าอันหลัง ออกก่อน”

เรียกหลักการนี้ว่า

“LIFO = Last In, First Out”

Last In-First Out (LIFO)



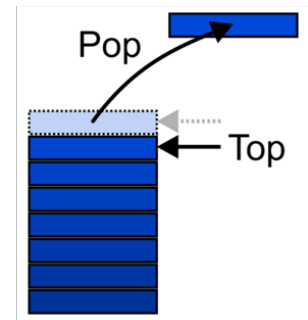
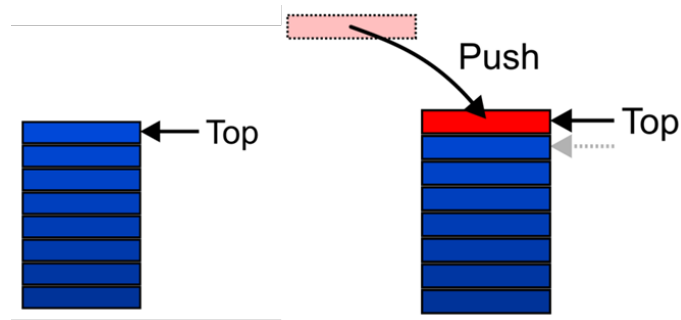
Stack Model

Stack มีด้านที่ใช้งานหลักเพียงด้านเดียว เรียกว่า “Top”

การเพิ่มข้อมูลทำที่ด้านบนการนำข้อมูลออกก็ทำที่ด้านบน

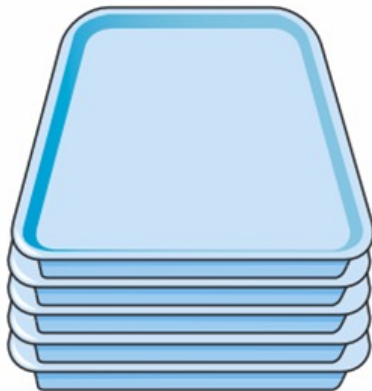
ดังนั้น Stack จึงมี Operations ดังนี้

- Top / Peek = ดูข้อมูลบนสุดโดยยังไม่ลบ
- Push = เพิ่มข้อมูลลงบน Stack
- Pop = นำข้อมูลบนสุดออก

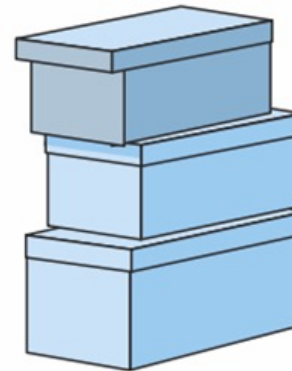


Real Life Example of Stack

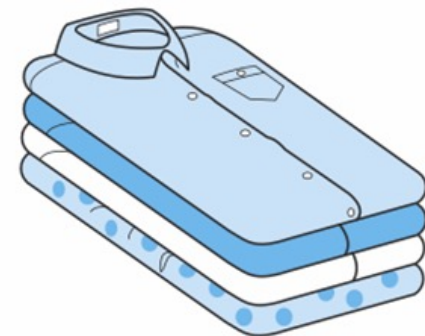
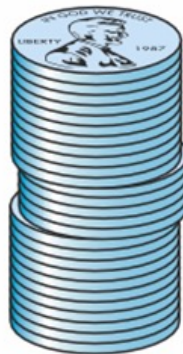
A stack of
cafeteria trays



A stack of
shoe boxes



A stack
of pennies



A stack of
neatly folded shirts

“สิ่งที่ใส่ทีหลัง มักจะถูกหยิบออกก่อน”

Example of Push

- เริ่มต้นจาก Stack ว่าง

- Push 10
- Push 20
- Push 30

- ผลลัพธ์

- 30 อยู่บนสุด
- 20 อยู่ถัดลงมา
- 10 อยู่ล่างสุด

Example of Pop

ถ้า Stack มี...

Top $\rightarrow$ 30, 20, 10

- เมื่อ Pop 1 ครั้ง
 - 30 จะถูกนำออก
- Stack ใหม่จะเป็น
 - Top $\rightarrow$ 20, 10

Example of Top

ถ้า Stack มี...

Top $\rightarrow$ 50, 40, 30

- คำสั่ง Top จะตอบว่า “50”
- แต่ข้อมูลยังคงอยู่ใน Stack

Stack Rule

- หลักสำคัญของ Stack
 - เพิ่มได้ที่ด้านบน
 - ลบได้ที่ด้านบน
 - มองเห็นข้อมูลสำคัญที่สุดคือข้อมูลบนสุด

Stack Vocabulary

- คำศัพท์สำคัญ
 - Stack = กองข้อมูล
 - Top = ตำแหน่งบนสุด
 - Push = เพิ่มข้อมูล
 - Pop = ลบข้อมูล
 - Empty Stack = Stack ว่าง



Stack in Memory Idea

ในระดับพื้นฐาน เราสามารถเก็บ Stack ด้วย Array ได้ และใช้ตัวแปรหนึ่งตัวเพื่อบอกตำแหน่ง Top
เช่น

- `array[ ]` เก็บข้อมูล
- `top` เก็บตำแหน่งล่าสุด

Stack Using Array

ตัวอย่างแนวคิด

ถ้า $\text{top} = -1$ หมายถึง Stack ว่าง

- เมื่อ Push ข้อมูลใหม่
 - เพิ่มค่า top
 - นำข้อมูลไปเก็บในตำแหน่งนั้น
- เมื่อ Pop
 - อ่านค่าที่ top
 - ลดค่า top ลง

Example with Array

- เริ่มต้น $\text{top} = -1$
- Push 8 $\text{top} = 0$
- Push 15 $\text{top} = 1$
- Push 25 $\text{top} = 2$

“แสดงว่า 25 คือข้อมูลบนสุด”



Basic C++ Idea for Stack

- ใช้อาร์เรย์เก็บข้อมูล
- ใช้ตัวแปร top ติดตามตำแหน่งบนสุด
- สร้างฟังก์ชัน push และ pop

Sample C++ Structure

</> C++

```
int stack[5];  
  
int top = -1;
```

อธิบาย

- stack[5] เก็บข้อมูลได้ 5 ตัว
- top = -1 หมายถึงยังไม่มีข้อมูล

Push Function Example

</> C++

```
void push(int x){  
    top++;  
    stack[top] = x;  
}
```

อธิบาย

- ขยับตำแหน่ง top ขึ้นก่อน
- เก็บข้อมูลใหม่ที่ตำแหน่ง top



Stack Overflow and Underflow

Overflow = พยายามเพิ่มข้อมูลทั้งที่ Stack เต็ม

Underflow = พยายามลบข้อมูลทั้งที่ Stack ว่าง

- Important Warning!

- ก่อน Push ควรเช็คกว่า “เต็มหรือยัง”
- ก่อน Pop ควรเช็คกว่า “ว่างหรือยัง”
- ถ้าไม่เช็ค อาจเกิดข้อผิดพลาดได้



Applications of Stack

- ตัวอย่างการใช้งาน
 - Undo และ Redo
 - Browser Back
 - การเรียกฟังก์ชันในโปรแกรม
 - ตรวจสอบวงเล็บเปิดปิด
 - แพลงนิพจน์ทางคณิตศาสตร์
 - แนวคิดเรื่องการตรวจวงเล็บและการประยุกต์ใช้ stack ปรากฏในชุดสไลด์เดิมเช่นกัน



Example Application: Undo

- เมื่อผู้ใช้พิมพ์งาน
- ระบบสามารถเก็บสถานะก่อนหน้าไว้ใน Stack
- เมื่อต้องการ Undo
- ก็นำสถานะล่าสุดออกมาใช้

Mini Exercise: Stack

กำหนดคำสั่งต่อไปนี้

Push 5

Push 9

Push 2

Pop

Push 7

คำถาม

- ข้อมูลบนสุดคืออะไร
- ข้อมูลใน Stack ตอนสุดท้ายมีอะไรบ้าง



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2 ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Section 2: Queue

What is Queue?

Queue คือ โครงสร้างข้อมูลที่ทำงานแบบ

“เข้าอันก่อน ออกก่อน”

เรียกหลักการนี้ว่า

“FIFO = First In, First Out”



Real Life Example of Queue

- ตัวอย่างในชีวิตจริง
 - ต่อคิวซื้อของ
 - รอขึ้นรถ
 - แถวหน้าโรงอาหาร
 - งานพิมพ์เอกสารหลายงาน
 - คนที่มาก่อน ควรได้รับบริการก่อน

Queue Model

- Queue มี 2 ด้านสำคัญ
 - Front = ด้านหน้า
 - Rear = ด้านหลัง

โดย

- เพิ่มข้อมูลทางด้านหลัง
- นำข้อมูลออกที่ด้านหน้า



Main Operations of Queue

- การทำงานพื้นฐานของ Queue
 - Enqueue = เพิ่มข้อมูลเข้าท้ายคิว
 - Dequeue = นำข้อมูลออกจากหน้าคิว
 - Front = ดูข้อมูลหน้าคิว

Example of Enqueue

- เริ่มจาก Queue ว่าง

Enqueue A

Enqueue B

Enqueue C

ผลลัพธ์:

Front $\rightarrow$ A, B, C $\leftarrow$ Rear

Example of Dequeue

- ถ้า Queue มี
 - Front $\rightarrow$ A, B, C
- เมื่อ Dequeue 1 ครั้ง
 - A จะออกก่อน
- Queue ใหม่จะเป็น
 - Front $\rightarrow$ B, C

Queue Rule

- หลักสำคัญของ Queue
 - เพิ่มด้านหลัง
 - ลบด้านหน้า
 - ผู้ที่มาก่อนจะออกก่อน

Stack vs Queue

Stack>

- เข้าและออกที่ด้านเดียว
- ข้อมูลล่าสุดออกก่อน

Queue>

- เข้าด้านหลัง ออกด้านหน้า
- ข้อมูลแรกออกก่อน



Queue Vocabulary

- Queue = คิว
- Front = ตำแหน่งหน้าสุด
- Rear = ตำแหน่งหลังสุด
- Enqueue = เพิ่มเข้าคิว
- Dequeue = ออกจากคิว



Queue Using Array

ในระดับพื้นฐาน Queue ก็สามารถสร้างด้วย Array ได้ โดยใช้ตัวแปร...

- front
- rear

เพื่อบอกว่าหน้าคิวและท้ายคิวอยู่ตรงไหน

Basic C++ Idea for Queue

</> C++

```
int queue[5];  
int front = 0;  
int rear = -1;
```

อธิบาย

- front ชี้ตำแหน่งหน้าคิว
- rear ชี้ตำแหน่งท้ายคิว
- เริ่มต้นยังไม่มีข้อมูล

Enqueue Example in C++

</> C++

```
void enqueue(int x){  
    rear++;  
    queue[rear] = x;  
}
```

อธิบาย

- ขยับ rear
- เก็บข้อมูลที่ตำแหน่ง rear

Deque Example in C++

</> C++

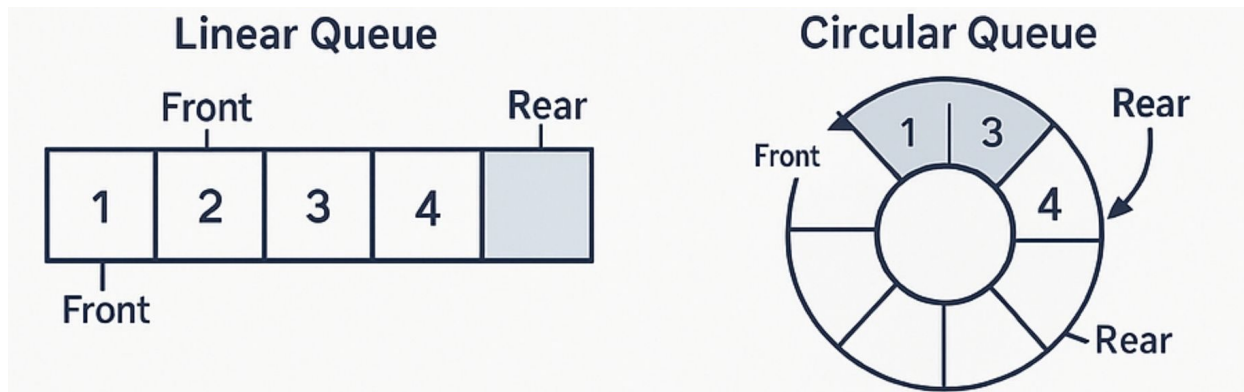
```
int dequeue(){  
    int x = queue[front];  
    front++;  
    return x;  
}
```

อธิบาย

- อ่านข้อมูลที่ front
- ขยับ front ไปข้างหน้า

Problem of Simple Queue

ถ้าใช้ Queue แบบง่ายด้วย Array แม้ข้อมูลด้านหน้าจะถูกลบไปแล้ว ช่องว่างด้านหน้า
อาจไม่ถูกนำกลับมาใช้ทันที จึงเกิดแนวคิด “Circular Queue”

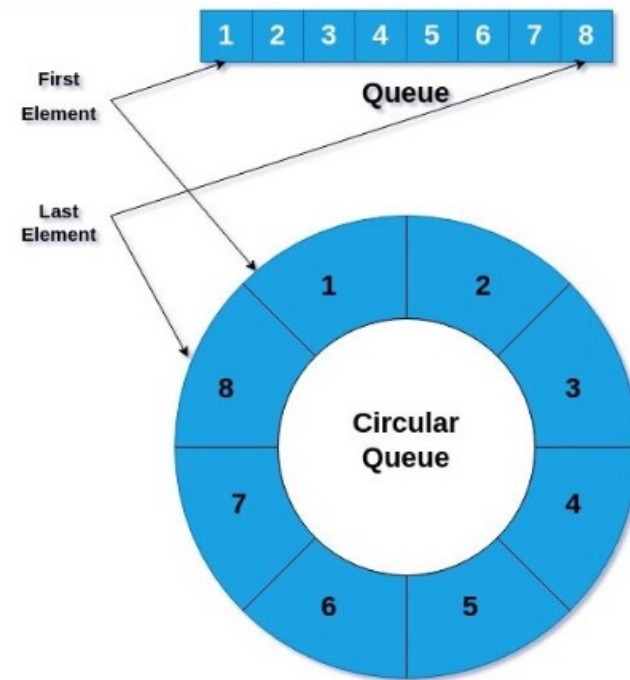


Circular Queue Idea

Circular Queue คือ การมองว่า Array

เชื่อมปลายกลับมาต้นได้

- เมื่อ rear ถึงช่องสุดท้าย
- ตำแหน่งถัดไปอาจกลับไปช่องแรก





Why Circular Queue is Better

- ข้อดี
 - ใช้พื้นที่ใน Array ได้คุ้มค่า
 - ลดการเสียช่องว่างด้านหน้า
 - เหมาะกับงานคิวที่มีการเข้าและออกตลอดเวลา



Applications of Queue

- ตัวอย่างการใช้งาน
 - ระบบคิวลูกค้า
 - ระบบพิมพ์เอกสาร
 - การจัดลำดับงานในระบบปฏิบัติการ
 - การจัดการคำขอบริการใน server
 - การประยุกต์ใช้ queue กับ client-server และงาน scheduling ก็มีในไฟล์เดิมเช่นกัน



Mini Exercise: Queue

กำหนดคำสั่งต่อไปนี้

Enqueue 10

Enqueue 20

Enqueue 30

Dequeue

Enqueue 40

คำถาม

- ข้อมูลหน้าคิวคืออะไร
- ข้อมูลในคิวตอนสุดท้ายมีอะไรบ้าง



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2 ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Section 3: Compare Stack and Queue

Similarities

- Stack และ Queue มีความเหมือนกันคือ
 - เป็นโครงสร้างข้อมูลเชิงเส้น
 - ใช้เก็บข้อมูลเป็นลำดับ
 - มีการเพิ่มและลบข้อมูล
 - สามารถสร้างด้วย Array หรือ Linked List ได้

Differences

Stack>

- LIFO
- เพิ่มและลบที่ด้านบน

Queue>

- FIFO
- เพิ่มด้านหลัง ลบด้านหน้า



When Should We Use Stack?

- ใช้ Stack เมื่อ...
 - ต้องการย้อนกลับ
 - ต้องการจัดการข้อมูลล่าสุดก่อน
 - ต้องการตรวจสอบความถูกต้องของสัญลักษณ์เปิดปิด
 - งานมีลักษณะซ้อนกันเป็นชั้น ๆ



When Should We Use Queue?

- ใช้ Queue เมื่อ
 - ต้องการความยุติธรรมตามลำดับเวลา
 - ใครมาก่อนออกก่อน
 - ต้องจัดการงานรอคิว
 - มีการร้องขอบริการต่อเนื่อง

Quick Summary Table

หัวข้อ	หลักการ	ตำแหน่งเพิ่ม	ตำแหน่งลบ	ตัวอย่างใช้งาน
Stack	LIFO	Top	Top	Undo
Queue	FIFO	Rear	Front	Waiting line



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2 ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Section 4: Linked List



Why Do We Need Linked List?

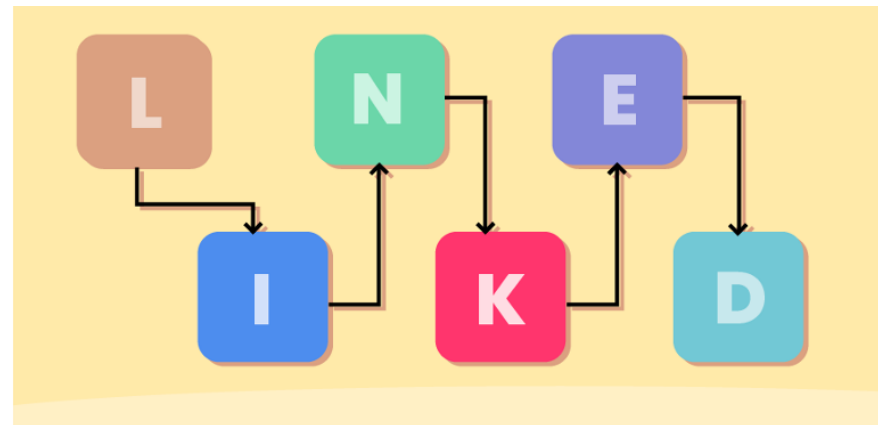
- ถ้าใช้ Array
 - ขนาดมักถูกกำหนดล่วงหน้า
 - การแทรกหรือลบตรงกลางทำได้ไม่สะดวก
 - บางครั้งต้องเลื่อนข้อมูลหลายตำแหน่ง
- จึงเกิดแนวความคิดการเก็บข้อมูลแบบเชื่อมโยงกัน
 - แนวคิดปัญหาของ array ก่อนเข้าสู่ linked list มีอยู่ในไฟล์ linked list เดิม

What is Linked List?

Linked List คือ โครงสร้างข้อมูล

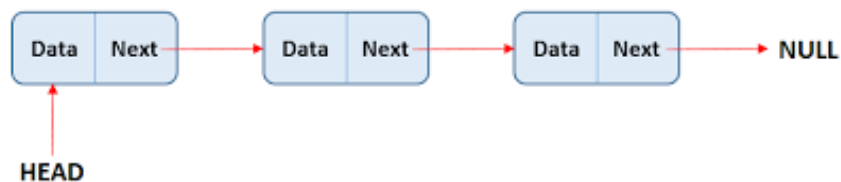
ที่เก็บข้อมูลเป็นข้อ ๆ

- แต่ละข้อเรียกว่า Node
- โดยแต่ละ node จะเชื่อมต่อไปยัง node ถัดไป



What is a Node?

- Node หนึ่งตัวประกอบด้วย
 - ข้อมูล
 - ตัวอ้างอิงไปยัง node ถัดไป
- หรือพูดง่าย ๆ Node คือ กล่องข้อมูลที่มีลูกศรชี้ไปข้างหน้า

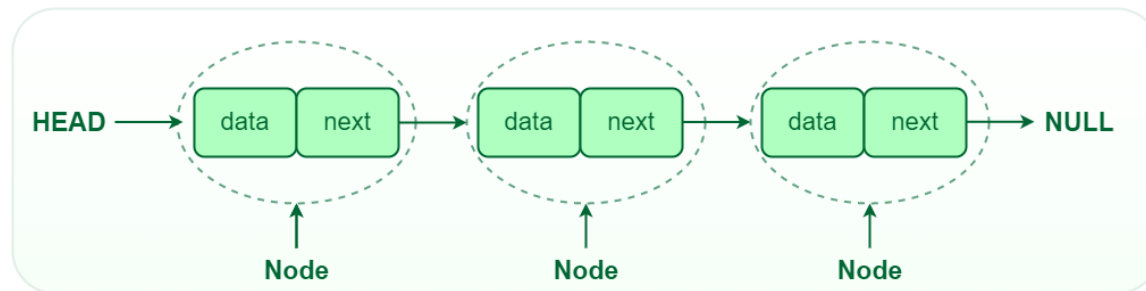


Node Structure

แนวคิดพื้นฐานของ node

- data
- next

ตัวอย่าง: [data | next]





Head Pointer

- Linked List ต้องมีตัวชี้เริ่มต้น เรียกว่า **Head**
- Head จะชี้ไปยัง node ตัวแรกของ list
- ถ้า list ว่าง head จะเป็น NULL



Example of Linked List

Head $\rightarrow$ [10|.] $\rightarrow$ [20|.] $\rightarrow$ [30|NULL]

ความหมาย

- 1) เริ่มจาก head
- 2) node แรกเก็บ 10
- 3) node ถัดไปเก็บ 20
- 4) node สุดท้ายเก็บ 30 และชี้ไปที่ NULL



Why is it Called Linked List?

“เพราะข้อมูลแต่ละตัวไม่ได้อยู่ติดกันแบบ Array แต่เชื่อมกันผ่านตัวชี้
จึงเรียกว่า list ที่เชื่อมโยงกัน”

Array vs Linked List

Array>

- ข้อมูลอยู่ติดกัน
- เข้าถึงด้วย index ได้ง่าย
- ขนาดมักกำหนดล่วงหน้า

Linked List>

- ข้อมูลเชื่อมกันด้วย pointer
- ขยายขนาดได้ยืดหยุ่นกว่า
- ต้องเดินทีละ node

Basic C++ Node Example

</> C++

```
class Node{  
public:  
    int value;  
    Node* next;  
};
```

อธิบาย

- value เก็บข้อมูล
- next เก็บที่อยู่ของ node ถัดไป

Node with Constructor

</> C++

```
class Node{  
public:  
    int value;  
    Node* next;  
  
    Node(int v){  
        value = v;  
        next = NULL;  
    }  
};
```

อธิบาย:

เมื่อสร้าง node ใหม่ จะกำหนดค่าเริ่มต้น
ให้ next เป็น NULL

Creating Nodes

`</> C++`

```
Node* a = new Node(10);
```

```
Node* b = new Node(20);
```

```
Node* c = new Node(30);
```

อธิบาย:

เราได้ node 3 ตัว แต่ยังไม่ได้เชื่อมกัน

Linking Nodes

`</> C++`

```
Node* a = new Node(10);
```

```
Node* b = new Node(20);
```

```
Node* c = new Node(30);
```

ผลลัพธ์:

a เชื่อมไป b

b เชื่อมไป c

c ยังชี้เป็น NULL



Traversing a Linked List

- การเดินผ่าน linked list หมายถึง
 - 1) การเริ่มจาก head
 - 2) ค่อย ๆ เลื่อนไป node ถัดไปที่ละตัว จนกว่าจะถึง NULL
- แนวคิดการเดิน list แบบ `ptr = ptr->next`

Traversal Example

`</> C++`

```
Node* ptr = head;
while(ptr != NULL){
    cout << ptr->value << " ";
    ptr = ptr->next;
}
```

อธิบาย:

- 1) เริ่มที่ head
- 2) แสดงข้อมูล
- 3) เลื่อนไป node ถัดไป
- 4) หยุดเมื่อถึงปลาย list



Key Idea to Remember

“Linked List ไม่มี index แบบ Array โดยตรง ถ้าต้องการดูตัวที่ 3 หรือ 4

ต้องเดินผ่านจากจุดเริ่มต้นไปที่ละตัว”



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2 ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Section 5: Linked List Operations



Main Operations on Linked List

- การทำงานพื้นฐาน
 - Insert at front
 - Delete at front
 - Traverse
 - Search
 - Insert at position
 - Delete at position

Push Front

Push Front คือ การเพิ่ม node ใหม่ไว้หน้าสุดของ list

- ตัวอย่าง
 - เดิม: Head $\rightarrow$ 20 $\rightarrow$ 30
- เพิ่ม 10 ด้านหน้า
 - ใหม่: Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30



Push Front Logic

ขั้นตอน

- 1) สร้าง node ใหม่
- 2) ให้ node ใหม่ชี้ไปยัง head เดิม
- 3) เปลี่ยน head ให้ชี้มายัง node ใหม่



Push Front Code

</> C++

```
Node* h = new Node(value);
```

```
h->next = head;
```

```
head = h;
```

Pop Front

Pop Front คือ การลบ node ตัวแรกออกจาก list

- ตัวอย่าง
 - เดิม: Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30
- หลังลบ
 - ใหม่: Head $\rightarrow$ 20 $\rightarrow$ 30



Pop Front Logic

ขั้นตอน

- 1) ดูว่า head ปัจจุบันคือใคร
- 2) เลื่อน head ไปยัง node ถัดไป
- 3) node เดิมถือว่าถูกนำออก



Pop Front Code

</> C++

```
head = head->next;
```



Search in Linked List

“การค้นหาข้อมูลใน linked list ต้องเดินผ่าน node ทีละตัว แล้ว
เปรียบเทียบค่าทีละจุด”



Search Example

- ค้นหาเลข 30
 - Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ 40
- ตรวจสอบทีละตัว
 - 10 ไม่ใช่
 - 20 ไม่ใช่
 - 30 ใช่



Insert at Position

“ถ้าต้องการแทรกข้อมูลตรงกลาง ต้องหาตำแหน่งก่อนหน้าให้เจอ
แล้วเชื่อม node ใหม่เข้าไป”

Insert Example

เดิม: Head $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 6 $\rightarrow$ 7

ต้องการแทรก 5 ระหว่าง 4 กับ 6

ผลลัพธ์: Head $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 7



Delete at Position

“ถ้าต้องการลบข้อมูลตรงกลาง ต้องเก็บ node ก่อนหน้าไว้ แล้วข้าม node ที่ต้องการลบ
ไปยังตัวถัดไป”

Delete Example

เดิม: Head $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 7

ลบ 5

ผลลัพธ์: Head $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 6 $\rightarrow$ 7



Linked List Strengths

- ข้อดี
 - เพิ่มข้อมูลด้านหน้าได้ง่าย
 - ลบด้านหน้าได้ง่าย
 - ขนาดยืดหยุ่น
 - ไม่ต้องจองพื้นที่ติดกันทั้งหมด

Linked List Limitations

- ข้อจำกัด
 - เข้าถึงข้อมูลด้วย index โดยตรงไม่สะดวก
 - ต้องใช้ pointer
 - เขียนโค้ดยากกว่า array สำหรับผู้เริ่มต้น
 - ต้องระวังการเชื่อมต่อผิด



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2 ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Section 6: Stack and Queue using Linked List



Important Idea

“Stack และ Queue เป็น “แนวคิดการใช้งาน” ส่วน Linked List เป็น “วิธีเก็บข้อมูล”

ดังนั้น เราสามารถสร้าง Stack และ Queue ด้วย Linked List ได้”



Stack using Linked List

- ถ้าจะทำ Stack ด้วย Linked List เราควรให้
 - Push ที่ด้านหน้า
 - Pop ที่ด้านหน้า
 - เพราะทำได้ง่ายและเร็ว



Why Front is Good for Stack

“เมื่อใช้ linked list การเพิ่มและลบที่หน้าสุดทำได้สะดวกมาก จึงเหมาะกับ Stack ที่ทำงานกับด้านเดียว”



Queue using Linked List

ถ้าจะทำ Queue ด้วย Linked List มักใช้

- Enqueue ที่ด้านหลัง
- Dequeue ที่ด้านหน้า

เพราะตรงกับหลัก FIFO



Why Linked List Helps Queue

“ถ้าข้อมูลเพิ่มและลดตลอดเวลา Linked List จะยืดหยุ่นกว่า Array โดยเฉพาะ

เมื่อจำนวนข้อมูลไม่แน่นอน”

Simple Concept Diagram

Stack with Linked List

- Push Front
- Pop Front

Queue with Linked List

- Push Back
- Pop Front



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2 ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Section 7: Advanced Preview



Doubly Linked List

ถ้า node หนึ่งตัวเก็บ pointer ได้ 2 ทาง

- next
- prev

เราจะสามารถเดินหน้าและถอยหลังได้ เรียกว่า **Doubly Linked List**

Singly vs Doubly Linked List

- Singly Linked List

- เดินไปข้างหน้าได้ทางเดียว

- Doubly Linked List

- เดินได้ทั้งไปข้างหน้าและย้อนกลับ
- ใช้หน่วยความจำมากขึ้นเล็กน้อย



Why This Topic Matters

- หัวข้อเหล่านี้เป็นพื้นฐานสำคัญของ
 - Algorithm
 - Programming
 - Competitive coding
 - Software development
 - System design
 - Database and memory concepts



Practice 1: Identify the Structure

- สถานการณ์ต่อไปนี้ควรใช้ Stack หรือ Queue
 - การ Undo งาน
 - การต่อคิวซื้อบัตร
 - Browser Back
 - ระบบพิมพ์เอกสาร
 - เก็บประวัติการเรียกฟังก์ชัน



Practice 2: Stack Tracing

ให้ตามคำสั่ง

Push 4

Push 8

Pop

Push 2

Push 9

Pop

คำถาม

- ข้อมูลที่ถูกนำออกมีอะไรบ้าง
- Stack สุดท้ายเหลืออะไร



Practice 3: Queue Tracing

ให้ตามคำสั่ง

Enqueue 1

Enqueue 2

Enqueue 3

Dequeue

Enqueue 4

Dequeue

คำถาม

- ข้อมูลที่ถูกนำออกมีอะไรบ้าง
- Queue สุดท้ายเหลืออะไร



Practice 4: Linked List Reading

จาก Head $\rightarrow$ 11 $\rightarrow$ 22 $\rightarrow$ 33 $\rightarrow$ NULL

คำถาม

- head ชี้ไปที่ค่าใด
- node ตัวสุดท้ายชี้ไปที่อะไร
- ถ้าจะเดินผ่านทุกตัว ต้องเริ่มจากอะไร



Practice 5: Predict the Result

เดิม Head $\rightarrow 5 \rightarrow 10 \rightarrow 15$

คำถาม

- ถ้า `push_front(2)` จะได้ผลลัพธ์เป็นอะไร
- ถ้า `pop_front()` ต่อก 1 ครั้ง จะเหลืออะไร

Common Mistakes

- ข้อผิดพลาดที่พบบ่อย
 - สับสนระหว่าง LIFO กับ FIFO
 - เข้าใจว่า Stack และ Queue เหมือนกัน
 - คิดว่า Linked List มี index ตรง ๆ แบบ Array
 - ลืมว่า node สุดท้ายต้องชี้เป็น NULL
 - สับสน front กับ rear



Memory Trick

- Stack = ล่าสุดออกก่อน
- Queue = มาก่อนออกก่อน
- Linked List = ข้อมูลต่อกันเป็นข้อ ๆ



Final Summary

- Stack ใช้หลัก LIFO
- Queue ใช้หลัก FIFO
- Linked List ใช้ node เชื่อมต่อกันด้วย pointer
- Stack และ Queue สร้างได้ทั้งจาก Array และ Linked List
- การเลือกใช้ขึ้นอยู่กับลักษณะปัญหา